

案件资料存储与关联方案



2026年03月14日



v1.0



案件资料存储与关联方案

文档版本：v1.0

编写日期：2026 年 3 月 14 日

开发负责人：王舟

一、整体架构

1.1 资料分类

资料类型	说明	示例
原始证据	当事人提供的原始文件	合同.pdf、录音.mp3、聊天记录.png
解析结果	从原始文件提取的文本和元数据	文字内容、页码、时间戳
衍生资料	系统生成的资料	证据清单.xlsx、时间轴图.png
锚定信息	原始文件与节点的关联关系	节点→页码/时间戳映射

1.2 数据流向

当事人提供原始证据



上传系统



原始文件存储

storage/files/{case_id}/{uuid}.pdf

↓
解析处理



解析结果存储	
storage/parsed/{case_id}/{id}.json	
database: evidence_files 表	



创建节点 + 关联



关联关系存储	
database: node_evidence_links 表	
锚定信息: 页码/时间戳/坐标	



导出资料



导出资料存储	
storage/exports/{case_id}/{name}	
证据清单、时间轴图、案件报告	

二、文件存储设计

2.1 目录结构

```
storage/
├── files/                                # 原始证据文件
│   ├── {case_id}/
│   │   ├── {uuid}.pdf                  # 原始 PDF
│   │   ├── {uuid}.mp3                  # 原始录音
│   │   ├── {uuid}.png                  # 原始图片
│   │   └── ...
│   └── ...
├── parsed/                              # 解析结果 (JSON)
│   ├── {case_id}/
│   │   ├── {evidence_id}.json          # 解析后的文本和锚点
│   │   └── ...
│   └── ...
```

```

└─ thumbnails/                                # 缩略图/预览图
    └─ {case_id}/
        └─ {evidence_id}_p1.jpg # 第 1 页缩略图
            └─ ...
└─ exports/                                    # 导出资料
    └─ {case_id}/
        └─ 证据清单_20260314.xlsx
        └─ 时间轴图_20260314.png
        └─ 案件报告_20260314.pdf
└─ temp/                                       # 临时文件（定期清理）
    └─ uploads/
        └─ {session_id}/
            └─ ...

```

2.2 文件命名规则

```

// src/utils/file_naming.rs
use uuid::Uuid;
use chrono::Utc;
use std::path::Path;

/// 生成存储文件名
pub fn generate_stored_name(original_name: &str) -> String {
    let uuid = Uuid::new_v4();
    let ext = Path::new(original_name)
        .extension()
        .and_then(|s| s.to_str())
        .unwrap_or("bin");

    // 格式: {uuid}.{ext}
    format!("{}.{}", uuid, ext.to_lowercase())
}

/// 生成解析结果文件名
pub fn generate_parsed_name(evidence_id: &str) -> String {
    format!("{}.json", evidence_id)
}

/// 生成导出文件名
pub fn generate_export_name(case_id: &str, doc_type: &str) -> String {
    let date = Utc::now().format("%Y%m%d_%H%M%S");
    format!("{_}_{_}.{}", case_id, doc_type, date,
        match doc_type {

```

```

        "evidence_list" => "xlsx",
        "timeline" => "png",
        "report" => "pdf",
        _ => "pdf",
    })
}

```

2.3 存储路径示例

```

storage/files/case_123/
├─ a1b2c3d4-e5f6-7890-abcd-ef1234567890.pdf      # 合同.pdf
├─ b2c3d4e5-f6a7-8901-bcde-f12345678901.mp3      # 录音.mp3
└─ c3d4e5f6-a7b8-9012-cdef-123456789012.png      # 聊天记录.png

storage/parsed/case_123/
├─ evidence_001.json    # 合同解析结果
├─ evidence_002.json    # 录音解析结果
└─ evidence_003.json    # 图片解析结果

storage/exports/case_123/
├─ case_123_evidence_list_20260314_103000.xlsx
├─ case_123_timeline_20260314_103000.png
└─ case_123_report_20260314_103000.pdf

```

三、数据库设计

3.1 核心表结构

```

-- 证据文件表
CREATE TABLE evidence_files (
    -- 主键
    id          TEXT PRIMARY KEY,

    -- 关联信息
    case_id     TEXT NOT NULL REFERENCES cases(id),

    -- 文件信息
    original_name TEXT NOT NULL,      -- 原始文件名: 合同.pdf
    stored_name  TEXT NOT NULL,      -- 存储文件名: uuid.pdf

```

```

file_type      TEXT NOT NULL,      -- MIME 类型: application/pdf
file_size      INTEGER NOT NULL,   -- 文件大小 (字节)

-- 存储路径
storage_path   TEXT NOT NULL,      -- 相对路径: files/case_id/uuid.pdf
storage_hash   TEXT,               -- 文件哈希 (SHA256, 用于去重)

-- 解析结果
parsed_text    TEXT,               -- 提取的完整文本
parsed_json    TEXT,               -- 结构化解析结果 (JSON)
page_count     INTEGER,            -- 页数 (PDF/图片)
duration       INTEGER,            -- 时长 (音频/视频, 秒)

-- OCR 信息
ocr_status     TEXT,               -- OCR 状态: pending/done/failed
ocr_confidence REAL,               -- OCR 置信度

-- 元数据
metadata       JSON,               -- 扩展元数据
tags           TEXT,               -- 标签: 逗号分隔

-- 状态
status         TEXT DEFAULT 'active', -- 状态: active/deleted/archived

-- 时间
uploaded_by    TEXT NOT NULL,      -- 上传人 ID
uploaded_at    DATETIME DEFAULT CURRENT_TIMESTAMP,
parsed_at      DATETIME,

-- 索引
INDEX idx_case ON evidence_files(case_id),
INDEX idx_status ON evidence_files(status),
INDEX idx_hash ON evidence_files(storage_hash)
);

-- 节点 - 证据关联表 (含锚定信息)
CREATE TABLE node_evidence_links (
  -- 主键
  id            TEXT PRIMARY KEY,

  -- 关联关系
  node_id       TEXT NOT NULL REFERENCES event_nodes(id),
  evidence_id   TEXT NOT NULL REFERENCES evidence_files(id),

  -- 锚定信息
  anchor_type   TEXT NOT NULL,      -- page/timestamp/paragraph/coordinate
  anchor_data   JSON NOT NULL,      -- 锚定详情

```

```

-- 锚定详情字段 (冗余, 方便查询)
page_num          INTEGER,                -- 页码 (anchor_type=page 时)
timestamp_sec     REAL,                  -- 时间戳秒数 (anchor_type=timestamp 时)
paragraph_start   INTEGER,                -- 段落起始位置
paragraph_end     INTEGER,                -- 段落结束位置
bbox              TEXT,                  -- 坐标框: x1,y1,x2,y2

-- 备注
note              TEXT,                  -- 关联备注
created_by        TEXT NOT NULL,
created_at        DATETIME DEFAULT CURRENT_TIMESTAMP,

-- 索引
INDEX idx_node ON node_evidence_links(node_id),
INDEX idx_evidence ON node_evidence_links(evidence_id)
);

-- 导出记录表
CREATE TABLE export_records (
  id              TEXT PRIMARY KEY,
  case_id         TEXT NOT NULL REFERENCES cases(id),
  export_type     TEXT NOT NULL,          -- evidence_list/timeline/report
  file_name       TEXT NOT NULL,
  storage_path    TEXT NOT NULL,
  file_size       INTEGER,
  generated_by    TEXT NOT NULL,
  generated_at    DATETIME DEFAULT CURRENT_TIMESTAMP,

-- 包含的内容
node_ids         TEXT,                  -- 包含的节点 ID 列表 (JSON 数组)
evidence_ids     TEXT,                  -- 包含的证据 ID 列表 (JSON 数组)

  INDEX idx_case ON export_records(case_id)
);

```

3.2 锚定信息数据结构

```

// src/models/anchor.rs
use serde::{Deserialize, Serialize};

/// 锚定类型
#[derive(Debug, Clone, Serialize, Deserialize, PartialEq)]
#[serde(rename_all = "lowercase")]
pub enum AnchorType {
  /// 页码锚定 (PDF/图片)

```

```

    Page,
    /// 时间戳锚定 (音频/视频)
    Timestamp,
    /// 段落锚定 (文本)
    Paragraph,
    /// 坐标锚定 (图片区域)
    Coordinate,
}

/// 锚定数据
#[derive(Debug, Clone, Serialize, Deserialize)]
#[serde(tag = "type", content = "data")]
pub enum AnchorData {
    Page {
        page_num: u32,           // 页码 (从 1 开始)
        page_label: Option<String>, // 页脚标签 (如 "第 3 页")
    },
    Timestamp {
        seconds: f64,           // 秒数
        formatted: String,      // 格式化时间: HH:MM:SS
    },
    Paragraph {
        start: usize,           // 起始字符位置
        end: usize,             // 结束字符位置
        preview: String,        // 内容预览 (前 50 字)
    },
    Coordinate {
        x: f64,                 // 左上角 X
        y: f64,                 // 左上角 Y
        width: f64,             // 宽度
        height: f64,            // 高度
        page_num: Option<u32>,   // 所属页码 (如果是图片内的区域)
    },
}

/// 关联关系
#[derive(Debug, Clone, Serialize, Deserialize)]
pub struct EvidenceLink {
    pub id: String,
    pub node_id: String,
    pub evidence_id: String,
    pub evidence_name: String, // 冗余, 方便展示
    pub anchor_type: AnchorType,
    pub anchor_data: AnchorData,
    pub note: Option<String>,
    pub created_at: chrono::DateTime<chrono::Utc>,
}

```

3.3 解析结果JSON 结构

```
// storage/parsed/case_123/evidence_001.json
{
  "evidence_id": "evidence_001",
  "file_type": "application/pdf",
  "original_name": "采购合同.pdf",
  "parsed_at": "2026-03-14T10:30:00Z",

  // 完整文本
  "full_text": "甲方: XXX 公司\n乙方: XXX 公司\n签订日期: 2024 年 1 月 15 日\n...",

  // 分页信息
  "pages": [
    {
      "page_num": 1,
      "text": "采购合同\n\n甲方: XXX 公司\n乙方: XXX 公司\n...",
      "width": 595,
      "height": 842,
      "blocks": [
        {
          "type": "title",
          "text": "采购合同",
          "bbox": [100, 50, 495, 80]
        },
        {
          "type": "paragraph",
          "text": "甲方: XXX 公司",
          "bbox": [100, 100, 300, 120]
        }
      ]
    },
    {
      "page_num": 2,
      "text": "第三条 付款方式...\n",
      ...
    }
  ],

  // 提取的关键信息
  "extracted_entities": [
    {
      "type": "date",
      "value": "2024-01-15",
      "text": "2024 年 1 月 15 日",
      "page_num": 1,
    }
  ]
}
```



```

        "position": 45
    },
    {
        "type": "amount",
        "value": "1000000.00",
        "text": "人民币 100 万元",
        "page_num": 2,
        "position": 120
    },
    {
        "type": "party",
        "value": "XXX 公司",
        "text": "甲方: XXX 公司",
        "page_num": 1,
        "position": 20
    }
],

// 锚点索引 (用于快速跳转)
"anchor_index": {
    "by_page": {
        "1": {"start": 0, "end": 500},
        "2": {"start": 501, "end": 1000}
    },
    "by_entity": {
        "date_1": {"page": 1, "position": 45},
        "amount_1": {"page": 2, "position": 120}
    }
}
}

```

四、文件上传与解析流程

4.1 上传流程

```

// src/routers/files.rs
use axum::{extract::Multipart, http::StatusCode};
use tokio::fs::File;
use tokio::io::AsyncWriteExt;

/// 上传文件

```

```

pub async fn upload_file(
    State(state): State<Arc<AppState>>,
    ctx: AppContext,
    case_id: String,
    mut multipart: Multipart,
) -> Result<Json<EvidenceFile>> {
    // 1. 接收文件
    let field = multipart.next_field().await?;
    let original_name = field.file_name().unwrap_or("unknown").to_string();
    let file_type = field.content_type().unwrap_or("application/octet-stream").to_string();
    let data = field.bytes().await?;

    // 2. 生成存储路径
    let stored_name = generate_stored_name(&original_name);
    let storage_path = format!("files/{}/{}", case_id, stored_name);
    let full_path = state.config.storage_root.join(&storage_path);

    // 3. 确保目录存在
    tokio::fs::create_dir_all(full_path.parent().unwrap()).await?;

    // 4. 保存文件
    let mut file = File::create(&full_path).await?;
    file.write_all(&data).await?;

    // 5. 计算文件哈希（用于去重）
    let file_hash = calculate_sha256(&data);

    // 6. 创建数据库记录
    let evidence = state.db
        .create_evidence(EvidenceFile {
            id: generate_id(),
            case_id: case_id.clone(),
            original_name,
            stored_name,
            file_type,
            file_size: data.len() as i64,
            storage_path,
            storage_hash: Some(file_hash),
            uploaded_by: ctx.user_id,
            ..Default::default()
        })
        .await?;

    // 7. 异步解析（不阻塞响应）
    let parser = state.parser.clone();
    let db = state.db.clone();
    let evidence_id = evidence.id.clone();

```

```

    tokio::spawn(async move {
        match parser.parse_file(&evidence).await {
            Ok(result) => {
                let _ = db.update_parsed_result(&evidence_id, &result).await;
            }
            Err(e) => {
                tracing::error!("解析失败 {}: {}", evidence_id, e);
                let _ = db.update_parse_status(&evidence_id, "failed").await;
            }
        }
    });

    Ok(Json(evidence))
}

```

4.2 解析流程

```

// src/services/parser.rs
use serde_json::json;

pub struct FileParser {
    storage_root: PathBuf,
}

impl FileParser {
    /// 解析文件
    pub async fn parse_file(&self, evidence: &EvidenceFile) -> Result<ParsedResult> {
        let file_path = self.storage_root.join(&evidence.storage_path);

        let result = match evidence.file_type.as_str() {
            "application/pdf" => self.parse_pdf(&file_path).await?,
            t if t.starts_with("image/") => self.parse_image(&file_path).await?,
            t if t.starts_with("audio/") => self.parse_audio(&file_path).await?,
            "application/vnd.openxmlformats-officedocument.spreadsheetml.sheet" |
            "application/vnd.ms-excel" => self.parse_excel(&file_path).await?,
            _ => self.parse_generic(&file_path).await?,
        };

        // 保存解析结果
        let json_path = self.storage_root
            .join("parsed")
            .join(&evidence.case_id)
            .join(format!("{}", evidence.id));

        tokio::fs::create_dir_all(json_path.parent().unwrap()).await?;
    }
}

```

```

        tokio::fs::write(&json_path, serde_json::to_string_pretty(&result)?).await?;

        Ok(result)
    }

    /// 解析 PDF
    async fn parse_pdf(&self, path: &Path) -> Result<ParsedResult> {
        let doc = PdfDocument::open(path)?;

        let mut pages = Vec::new();
        let mut full_text = String::new();

        for (i, page) in doc.pages().enumerate() {
            let text = page.text();
            let blocks = self.extract_blocks(&page);

            pages.push(json!({
                "page_num": i + 1,
                "text": text,
                "width": page.width(),
                "height": page.height(),
                "blocks": blocks,
            }));

            full_text.push_str(&text);
        }

        Ok(ParsedResult {
            file_type: "application/pdf".to_string(),
            full_text,
            page_count: Some(doc.page_count()),
            pages,
            extracted_entities: self.extract_entities(&full_text),
            anchor_index: self.build_anchor_index(&pages),
        })
    }

    /// 解析音频
    async fn parse_audio(&self, path: &Path) -> Result<ParsedResult> {
        // 调用 FunASR 进行语音识别
        let transcription = self.asr.transcribe(path).await?;

        let segments = transcription.segments
            .iter()
            .map(|seg| json!({
                "start": seg.start,
                "end": seg.end,
            }));
    }

```

```

        "text": seg.text,
    )))
    .collect();

Ok(ParsedResult {
    file_type: "audio/mpeg".to_string(),
    full_text: transcription.text,
    duration: Some(transcription.duration as i64),
    pages: vec![json!({
        "segments": segments
    })],
    extracted_entities: self.extract_entities(&transcription.text),
    anchor_index: self.build_timestamp_index(&transcription.segments),
})
}
}

```

4.3 锚定信息创建

```

// src/routers/nodes.rs

/// 关联证据到节点（带锚定）
pub async fn link_evidence(
    State(state): State<Arc<AppState>>,
    ctx: AppContext,
    Path((node_id, evidence_id)): Path<(String, String)>,
    Json(input): Json<LinkEvidenceInput>,
) -> Result<Json<EvidenceLink>> {
    // input.anchor_type: "page" | "timestamp" | "paragraph" | "coordinate"
    // input.anchor_data: 根据类型不同而不同

    // 1. 验证节点和证据存在
    let node = state.db.get_node(&node_id).await?;
    let evidence = state.db.get_evidence(&evidence_id).await?;

    // 2. 创建关联
    let link = state.db
        .create_evidence_link(EvidenceLink {
            id: generate_id(),
            node_id: node_id.clone(),
            evidence_id: evidence_id.clone(),
            evidence_name: evidence.original_name.clone(),
            anchor_type: input.anchor_type,
            anchor_data: input.anchor_data,
            note: input.note,
        })
        .await?;
    Ok(Json(link))
}

```

```

        created_by: ctx.user_id,
    })
    .await?;

// 3. 记录日志
log_operation!(
    ctx,
    action: ActionType::Link,
    module: ModuleType::Node,
    target_type: "node_evidence_link",
    target_id: link.id.clone(),
    target_title: Some(format!("{}", node.title, evidence.original_name)),
    old: None,
    new: Some(json!(&link))
);

Ok(Json(link))
}

```

五、双向跳转实现

5.1 节点 → 原始证据

```

// src/services/jump.rs

/// 获取节点的跳转目标
pub async fn get_jump_targets(
    db: &Database,
    node_id: &str,
) -> Result<Vec<JumpTarget>> {
    // 1. 获取节点关联的所有证据
    let links = db.get_node_links(node_id).await?;

    let mut targets = Vec::new();

    for link in links {
        let evidence = db.get_evidence(&link.evidence_id).await?;

        // 2. 根据锚定类型构建跳转目标
        let target = match link.anchor_type {
            "page" => {

```

```

        let page_num = link.anchor_data["page_num"].as_u64().unwrap();
        JumpTarget {
            evidence_id: evidence.id,
            evidence_name: evidence.original_name,
            file_type: evidence.file_type,
            storage_path: evidence.storage_path,
            location: JumpLocation::Page(page_num),
        }
    }
    "timestamp" => {
        let seconds = link.anchor_data["seconds"].as_f64().unwrap();
        JumpTarget {
            evidence_id: evidence.id,
            evidence_name: evidence.original_name,
            file_type: evidence.file_type,
            storage_path: evidence.storage_path,
            location: JumpLocation::Timestamp(seconds),
        }
    }
    "paragraph" => {
        let start = link.anchor_data["start"].as_u64().unwrap() as usize;
        let end = link.anchor_data["end"].as_u64().unwrap() as usize;
        JumpTarget {
            evidence_id: evidence.id,
            evidence_name: evidence.original_name,
            file_type: evidence.file_type,
            storage_path: evidence.storage_path,
            location: JumpLocation::Paragraph(start, end),
        }
    }
    _ => continue,
};

    targets.push(target);
}

Ok(targets)
}

/// 跳转位置
#[derive(Debug, Clone)]
pub enum JumpLocation {
    Page(u32),
    Timestamp(f64),
    Paragraph(usize, usize),
    Coordinate(f64, f64, f64, f64),
}

```

5.2 前端跳转组件

```
// frontend/src/components/EvidencePreview.rs

#[component]
pub fn EvidencePreview(evidence_id: String, jump_location: JumpLocation) -> Element {
    let evidence = use_resource(move || fetch_evidence(&evidence_id));

    rsx! {
        div { class: "evidence-preview",
            match &*evidence.value() {
                Some(Ok(e)) => {
                    let file_type = e.file_type.as_str();

                    if file_type == "application/pdf" {
                        rsx! {
                            PdfViewer {
                                src: format!("/api/v1/files/{}/download", e.id),
                                initial_page: match jump_location {
                                    JumpLocation::Page(p) => p,
                                    _ => 1,
                                },
                                highlight: match &jump_location {
                                    JumpLocation::Paragraph(start, end) => {
                                        Some((*start, *end))
                                    },
                                    _ => None,
                                },
                            },
                        }
                    } else if file_type.starts_with("audio/") {
                        rsx! {
                            AudioPlayer {
                                src: format!("/api/v1/files/{}/download", e.id),
                                initial_time: match jump_location {
                                    JumpLocation::Timestamp(t) => t,
                                    _ => 0.0,
                                },
                            },
                        }
                    } else if file_type.starts_with("image/") {
                        rsx! {
                            ImageViewer {
                                src: format!("/api/v1/files/{}/download", e.id),
                                bbox: match &jump_location {
                                    JumpLocation::Coordinate(x, y, w, h) => {
```



```

        Some((*x, *y, *w, *h))
    }
    _ => None,
  },
}
}
} else {
  rsx! {
    div { class: "unsupported",
      "不支持预览的文件类型"
    }
  }
}
}
_ => rsx! { div { "加载中..." } }
}
}
}
}
}

```

六、导出资料关联

6.1 证据清单导出

```
// src/services/export.rs

/// 导出证据清单
pub async fn export_evidence_list(
    db: &Database,
    case_id: &str,
    node_ids: Option<Vec<String>>,
) -> Result<ExportRecord> {
    // 1. 获取案件信息
    let case_info = db.get_case(case_id).await?;

    // 2. 获取节点列表
    let nodes = if let Some(ids) = node_ids {
        db.get_nodes_by_ids(&ids).await?
    } else {
        db.get_all_nodes(case_id).await?
    };
}
```

```

// 3. 获取每个节点关联的证据
let mut evidence_map = HashMap::new();
for node in &nodes {
    let links = db.get_node_links(&node.id).await?;
    for link in links {
        evidence_map.entry(link.evidence_id.clone())
            .or_insert_with(Vec::new)
            .push(node.clone());
    }
}

// 4. 获取证据详情
let evidence_list = Vec::new();
for (evidence_id, related_nodes) in evidence_map {
    let evidence = db.get_evidence(&evidence_id).await?;
    evidence_list.push(ExportEvidence {
        original_name: evidence.original_name,
        file_type: evidence.file_type,
        page_count: evidence.page_count,
        related_nodes: related_nodes.iter().map(|n| n.title.clone()).collect(),
        upload_date: evidence.uploaded_at,
    });
}

// 5. 生成 Excel
let excel_data = generate_evidence_list_excel(&case_info, &evidence_list);

// 6. 保存导出文件
let file_name = generate_export_name(case_id, "evidence_list");
let storage_path = format!("exports/{}/{}", case_id, file_name);
let full_path = CONFIG.storage_root.join(&storage_path);

tokio::fs::create_dir_all(full_path.parent().unwrap()).await?;
tokio::fs::write(&full_path, excel_data).await?;

// 7. 创建导出记录
let record = db
    .create_export_record(ExportRecord {
        id: generate_id(),
        case_id: case_id.to_string(),
        export_type: "evidence_list".to_string(),
        file_name,
        storage_path,
        file_size: Some(excel_data.len() as i64),
        generated_by: "system".to_string(),
        node_ids: Some(json!(nodes.iter().map(|n| &n.id).collect:::<Vec<_>>()))
    })

```

```

        evidence_ids: Some(json!(evidence_map.keys().collect::<Vec<_>>(())),
    })
    .await?;

    Ok(record)
}

```

6.2 导出记录查询

```

/// 获取案件的导出历史
pub async fn get_export_history(
    db: &Database,
    case_id: &str,
) -> Result<Vec<ExportRecord>> {
    let records = sqlx::query_as::<_, ExportRecord>(
        r#"
        SELECT * FROM export_records
        WHERE case_id = ?
        ORDER BY generated_at DESC
        LIMIT 50
        "#,
    )
    .bind(case_id)
    .fetch_all(&db.pool)
    .await?;

    Ok(records)
}

/// 下载导出文件
pub async fn download_export(
    db: &Database,
    export_id: &str,
) -> Result<FileResponse> {
    let record = db.get_export_record(export_id).await?;
    let file_path = CONFIG.storage_root.join(&record.storage_path);

    Ok(FileResponse::new(file_path, record.file_name))
}

```

七、数据关系图



```
| └─ {case_id}/  
|   └─ {uuid}.pdf      ← stored_name  
|   └─ {uuid}.mp3  
|   └─ {uuid}.png
```

八、安全与权限

8.1 文件访问控制

```
// src/middleware/file_auth.rs  
  
/// 验证文件访问权限  
pub async fn verify_file_access(  
    ctx: &AppContext,  
    case_id: &str,  
) -> Result<()> {  
    // 1. 验证用户有案件访问权  
    let has_access = ctx  
        .db  
        .check_case_access(&ctx.user_id, case_id)  
        .await?;  
  
    if !has_access {  
        return Err(Error::Forbidden("无权访问该案件的证据文件"));  
    }  
  
    Ok(())  
}  
  
/// 生成带签名的下载链接（有效期 1 小时）  
pub fn generate_signed_url(  
    evidence_id: &str,  
    user_id: &str,  
    expires_in: Duration,  
) -> String {  
    let expires = Utc::now() + expires_in;  
    let signature = calculate_signature(evidence_id, user_id, &expires);  
  
    format!(  
        "/api/v1/files/{}/download?signature={}&expires={}",  
        evidence_id, signature, expires.to_rfc3339()
```

```

        evidence_id,
        signature,
        expires.timestamp()
    )
}

```

8.2 文件完整性校验

```

/// 验证文件完整性（防止篡改）
pub async fn verify_file_integrity(
    db: &Database,
    evidence_id: &str,
) -> Result<bool> {
    let evidence = db.get_evidence(evidence_id).await?;

    // 重新计算文件哈希
    let file_path = CONFIG.storage_root.join(&evidence.storage_path);
    let current_hash = calculate_file_sha256(&file_path).await?;

    // 与存储的哈希对比
    Ok(evidence.storage_hash == Some(current_hash))
}

```

九、备份与归档

9.1 定期备份

```

/// 每日备份
pub async fn daily_backup() {
    let backup_dir = CONFIG.backup_root.join(Utc::now().format("%Y%m%d").to_string());
    tokio::fs::create_dir_all(&backup_dir).await.unwrap();

    // 1. 备份数据库
    let db_backup = backup_dir.join("legal_minds.db");
    tokio::fs::copy(CONFIG.db_path, &db_backup).await.unwrap();

    // 2. 备份存储文件（增量备份）
    let storage_backup = backup_dir.join("storage");
    sync_storage(&storage_backup).await.unwrap();
}

```

```
// 3. 上传到云存储 (可选)
upload_to_cloud(&backup_dir).await.unwrap();
}
```

9.2 归档策略

数据类型	归档周期	存储位置
原始证据	永久	主存储 + 云备份
解析结果	永久	主存储
导出资料	5 年	主存储 → 冷存储
操作日志	永久	主存储 → 归档表
临时文件	7 天	自动清理

十、总结

10.1 关键设计决策

问题	方案	理由
原始文件如何存储	文件系统 + 数据库记录	大文件不适合存数据库
如何关关节点与证据	关联表 + 锚定信息	支持双向跳转
解析结果存哪里	JSON 文件 + 数据库索引	灵活查询 + 快速检索
如何保证文件安全	哈希校验 + 签名链接	防篡改 + 防外链
导出资料如何管理	导出记录表	可追溯 + 可复用

10.2 数据量估算

数据类型	单案件估算	100 案件
原始证据	500MB（50 份文件）	50GB
解析结果	10MB（JSON）	1GB
缩略图	50MB	5GB
导出资料	10MB	1GB
合计	570MB	57GB

文档版本：v1.0

最后更新：2026 年 3 月 14 日

开发负责人：王舟

邮箱：main@mails.wedevs.org

电话：15378391447

开发负责人：王舟 | 电话：15378391447 | 邮箱：main@mails.wedevs.org